

Build the ML Pipeline

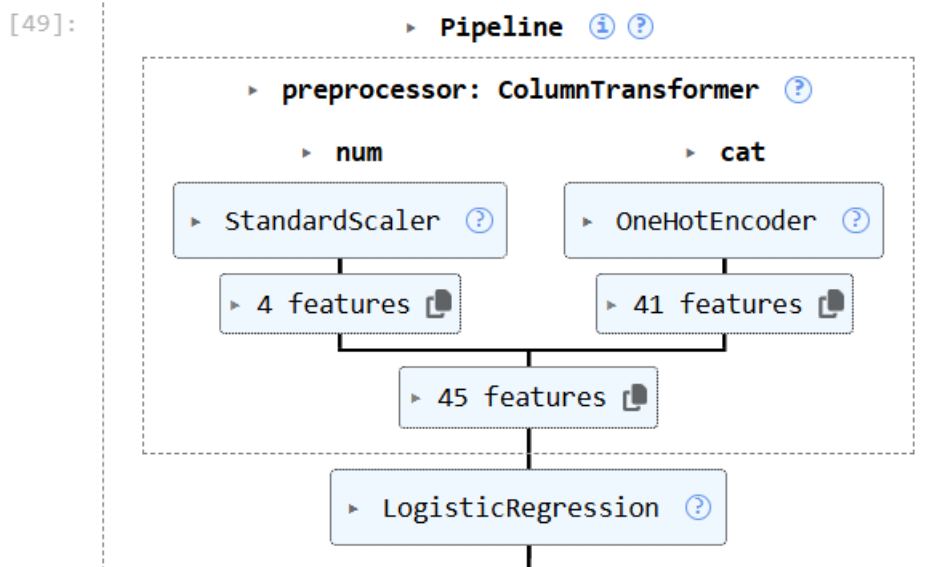
Logistic Regression

```
[47]: from sklearn.pipeline import Pipeline
      from sklearn.linear_model import LogisticRegression

      logistic_model = Pipeline([
          ("preprocessor", preprocessor),
          ("classifier", LogisticRegression(max_iter=1000))
      ])
```

```
[48]: # Now we train it
```

```
[49]: logistic_model.fit(X_train, y_train)
```



```
[50]: # Make predictions
```

```
[51]: y_pred = logistic_model.predict(X_test)
```

```
[52]: y_pred[:20]
```

```
[52]: array(['No', 'Yes', 'No', 'No', 'No', 'Yes', 'No', 'No', 'No', 'No', 'No',
        'No', 'No', 'Yes', 'No', 'No', 'No', 'Yes', 'No', 'No'],
        dtype=object)
```

```
[53]: # Also get churn probabilities
```

```
[54]: y_proba = logistic_model.predict_proba(X_test)[: , 1]
```

```
[55]: logistic_model.classes_
```

```
[55]: array(['No', 'Yes'], dtype=object)
```

```
[56]: # Classification Report
```

```
[57]: from sklearn.metrics import accuracy_score, classification_report
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

Accuracy: 0.8055358410220014

Classification Report:

	precision	recall	f1-score	support
No	0.85	0.89	0.87	1035
Yes	0.66	0.56	0.60	374
accuracy			0.81	1409
macro avg	0.75	0.73	0.74	1409
weighted avg	0.80	0.81	0.80	1409

```
[58]: # Confusion Matrix
```

```
[59]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

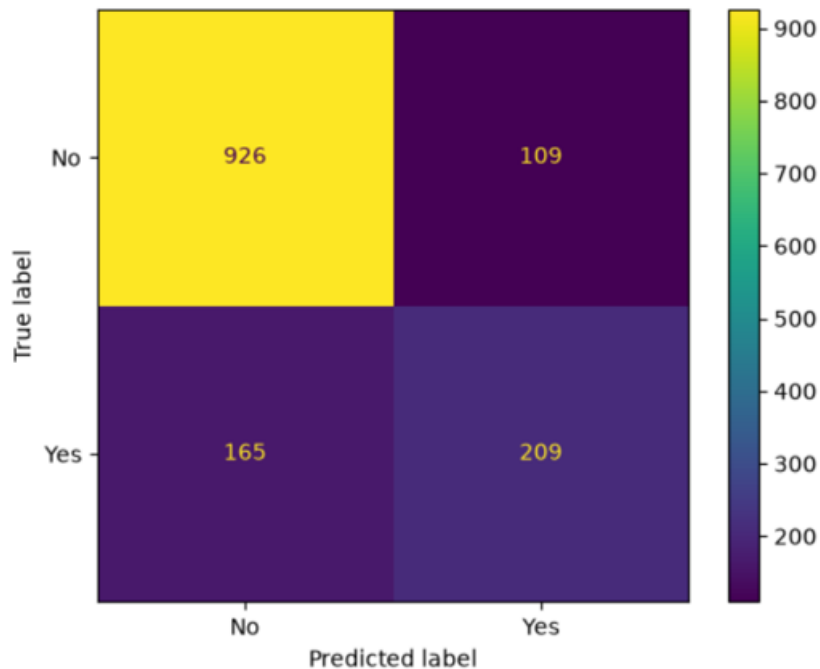
```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
[[926 109]
 [165 209]]
```

```
[60]: ConfusionMatrixDisplay(
      confusion_matrix=cm,
      display_labels=logistic_model.classes_
    ).plot()
```

```
[60]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x21abb2ad940>
```



```
[61]: # metric: ROC-AUC.
```

```
[61]: # metric: ROC-AUC.
```

```
[62]: from sklearn.metrics import roc_auc_score
```

```
auc = roc_auc_score(y_test, y_proba)
```

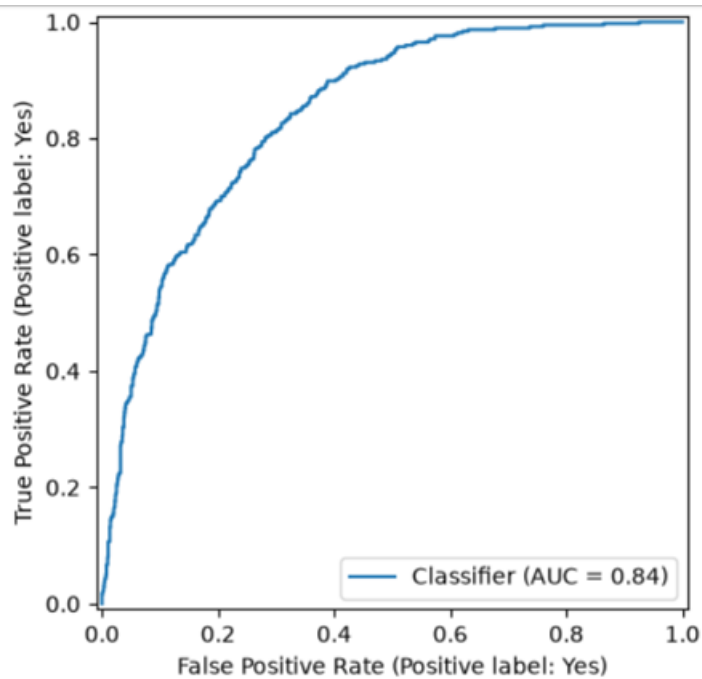
```
print("ROC-AUC:", auc)
```

```
ROC-AUC: 0.8421349040274871
```

```
[64]: from sklearn.metrics import RocCurveDisplay
```

```
RocCurveDisplay.from_predictions(  
    y_test,  
    y_proba,  
    pos_label="Yes"  
)
```

```
[64]: <sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x21ac1323770>
```



```
[66]: from sklearn.metrics import precision_score, recall_score, f1_score
import numpy as np
for threshold in [0.5, 0.4, 0.3, 0.2]:
    y_pred_threshold = np.where(
        y_proba >= threshold,
        "Yes",
        "No"
    )

    print(f"Threshold: {threshold}")
    print("Precision:", precision_score(y_test, y_pred_threshold, pos_label="Yes"))
    print("Recall:", recall_score(y_test, y_pred_threshold, pos_label="Yes"))
    print("F1:", f1_score(y_test, y_pred_threshold, pos_label="Yes"))
    print()
```

Threshold: 0.5
Precision: 0.6572327044025157
Recall: 0.5588235294117647
F1: 0.6040462427745664

Threshold: 0.4
Precision: 0.5681818181818182
Recall: 0.6684491978609626
F1: 0.6142506142506142

Threshold: 0.3
Precision: 0.5193370165745856
Recall: 0.7540106951871658
F1: 0.6150490730643402

Threshold: 0.2
Precision: 0.46715328467153283
Recall: 0.8556149732620321
F1: 0.6043437204910292